📚 Day 01 — DSA Notes (Java)

> **Topics Covered:** Loops, Number Theory, Factors, Primes, GCD, LCM, Digit Manipulation

🗂 Table of Contents

#	Problem	Concept
01	Print Reverse Numbers	For loop (countdown)
02	Print Even Numbers	Modulo operator
03	Print Odd Numbers	Modulo operator
04	Print Multiples of A and B	OR condition with modulo
05	Print Multiplication Table	For loop
06	Print Factors of N	Divisibility check
07	Print Sum (Count) of Factors	Count using loop
08	Check Prime or Not	Optimised prime check `i*i <= n`
09	Print Primes Between N and M	Range + PrimeCheck helper
10	Print First N Prime Numbers	While loop + counter
11	Print Nth Prime Number	Track last prime found
12	Print Multiples of N Till X	Loop with modulo filter
13	Print Common Multiples of A & B	(Placeholder / pending)

14	Print First N Common Multiples of A & B	While loop + LCM logic
15	Print Common Factors of N1 and N2	Loop up to `min(N1, N2)`
16	Find GCD	Euclidean subtraction method
17	Find LCM	Formula: `LCM = (N1 * N2) / GCD`
18	Count Digits	Divide by 10 repeatedly
19	Sum of Digits	Modulo 10 repeatedly
20	Sum of Even Digits	Filter `mod % 2 == 0`
21	Sum of Odd Digits	Filter `mod % 2 != 0`
22	Reverse a Number	Build string from last digits

Section 1: Loops & Basic Iteration

001 — Print Numbers in Reverse

Idea: Start from `n` and count down to `1`.

```
for (int i = n; i >= 1; i--) {
    System.out.print(i + " ");
}
```

> **Key Concept:** Decrement loop — `i--` goes from `n → 1`.

---

002 — Print Even Numbers from 1 to N

Idea: A number is **even** if `i % 2 == 0`.

```
for (int i = 1; i <= n; i++) {
    if (i % 2 == 0) {
        System.out.println(i);
    }
}
```

> **Key Concept:** `%` (modulo) gives the **remainder**. If remainder is 0 → even.

---

003 — Print Odd Numbers from 1 to N

Idea: A number is **odd** if `i % 2 != 0`.

```
for (int i = 1; i <= n; i++) {
    if (i % 2 != 0) {
        System.out.println(i);
    }
}
```

> **Key Concept:** Same pattern as even — flip the condition.

---

004 — Print Multiples of A or B up to N

Idea: Print numbers divisible by **A or B** (not necessarily both).

```
for (int i = 1; i <= n; i++) {
    if (i % a == 0 || i % b == 0) {
        System.out.println(i);
    }
}
```

> **Key Concept:** Use `||` (OR) when either condition qualifies.

---

005 — Print Multiplication Table of N

Idea: Loop `i` from 1 to 10 and print `n * i`.

```
for (int i = 1; i <= 10; i++) {
    System.out.println(n + " X " + i + " = " + n * i);
}
```

> **Key Concept:** Fixed-count loop (1 to 10). String concatenation formats the output.

Section 2: Factors

006 — Print All Factors of N

Definition: `i` is a factor of `n` if `n % i == 0`.

```
for (int i = 1; i <= n; i++) {
    if (n % i == 0) {
        System.out.println(i);
    }
}
```

> **Example:** Factors of 12 → 1, 2, 3, 4, 6, 12

---

007 — Count the Number of Factors of N

Idea: Same as above, but keep a **counter** instead of printing.

```
int count = 0;
for (int i = 1; i <= n; i++) {
    if (n % i == 0) {
```

```
            count++;
        }
    }
    System.out.println(count);
```

> **Note:** The file is named "Sum_Of_Factors" but actually **counts** factors. `count++` adds 1 per factor, not the factor's value.

---

015 — Print Common Factors of N1 and N2

Idea: A common factor divides **both** N1 and N2. Loop up to the smaller number.

```
int min = (N < X) ? N : X;

for (int i = 1; i <= min; i++) {
    if (N % i == 0 && X % i == 0) {
        System.out.println(i);
    }
}
```

> **Key Concept:** Common factors cannot exceed `min(N1, N2)`.

Section 3: Prime Numbers

Prime Number Theory

> A **prime number** is a number greater than 1 that has **no divisors other than 1 and itself**.
>
> Examples: 2, 3, 5, 7, 11, 13, ...

---

008 — Check If a Number Is Prime (Optimised)

```
public static boolean PrimeCheck(int n) {
    if (n <= 1) return false;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}
```

Why `i * i <= n` instead of `i <= n`?

If `n` has a factor larger than sqrt(n), the **other** factor must be smaller than sqrt(n) — so we'd have already found it!

> **Time Complexity:** O(sqrt(n)) instead of O(n) — much faster for large numbers.

n	Loop goes up to
100	10
10,000	100
1,000,000	1,000

---

009 — Print All Primes Between N and M

```
public static void Print(int N, int M) {
    for (int i = N; i <= M; i++) {
        if (PrimeCheck(i)) {
            System.out.println(i);
        }
    }
}
```

> **Pattern:** Reuse the `PrimeCheck` helper function.

---

010 — Print First N Prime Numbers

```
public static void Print(int count) {
    int num = 0;  // primes found so far
    int i = 1;
    while (num < count) {
        if (PrimeCheck(i)) {
            System.out.println(i);
            num++;
        }
        i++;
    }
}
```

> **Key Concept:** Use a **while loop** when you don't know how many iterations it will take. The loop stops when `count` primes have been found.

---

011 — Find the Nth Prime Number

```
public static int Nth_Prime(int count) {
    int num = 0;
    int i = 1;
    int lastPrime = 0;
    while (num < count) {
        if (PrimeCheck(i)) {
            lastPrime = i;  // keep updating the last pri
            num++;
        }
        i++;
    }
    return lastPrime;
}
```

> **Key Difference from #010:** Instead of printing each prime, we **store** the last prime. When the loop ends, `lastPrime` holds the Nth prime.

Section 4: Multiples

012 — Print Multiples of N Up to X

```
for (int i = N; i <= X; i++) {
    if (i % N == 0) {
        System.out.print(i + " ");
    }
}
```

> **Simpler alternative:** `for (int i = N; i <= X; i += N)` — step by N directly.

---

014 — Print Common Multiples of A and B up to X

Idea: A common multiple is divisible by **both** A and B.

```
int i = Math.max(N, N1); // start from the larger of the
while (i < X) {
    if (i % N == 0 && i % N1 == 0) {
        System.out.println(i);
        i++;
    }
    i++;
}
```

> **Mathematical insight:** All common multiples are multiples of `LCM(A, B)`.

Section 5: GCD & LCM

Theory

Term	Definition
GCD (Greatest Common Divisor)	Largest number that divides both N1 and N2
LCM (Least Common Multiple)	Smallest number that is divisible by both N1 and N2

Key Formula:

```
LCM(a, b) = (a x b) / GCD(a, b)
```

---

016 — Find GCD (Euclidean Subtraction Method)

```
public static int GCD(int N1, int N2) {
    while (N1 != N2) {
        if (N1 < N2) {
            N2 = N2 - N1;
        } else {
            N1 = N1 - N2;
        }
    }
    return N1;
}
```

How It Works (Example: GCD of 48 and 18):

```
48, 18 → 30, 18 → 12, 18 → 12, 6 → 6, 6   GCD = 6
```

Alternative — Modulo method (faster):

```
while (N2 != 0) {
    int temp = N2;
    N2 = N1 % N2;
    N1 = temp;
}
return N1;
```

---

017 — Find LCM

```
int gcd = GCD(N1, N2);
int lcm = (N1 * N2) / gcd;
System.out.println(lcm);
```

Formula: `LCM = (N1 x N2) / GCD(N1, N2)`

Example: LCM(4, 6) = (4 x 6) / GCD(4,6) = 24 / 2 = **12**

Section 6: Digit Manipulation

Core trick: Use `n % 10` to extract the **last digit**, and `n / 10` to remove it.

Operation	Effect
`n % 10`	Get last digit
`n / 10`	Remove last digit (shift right)

---

018 — Count the Number of Digits

```
public static int Counting(int n) {
    int count = 0;
    while (n != 0) {
        n = n / 10;
        count++;
    }
    return count;
}
```

> **Example:** n = 4567 → 456 → 45 → 4 → 0 (4 iterations → 4 digits)

---

019 — Sum of All Digits

```
public static int Add(int n) {
    int sum = 0;
    while (n != 0) {
        int mod = n % 10;   // extract last digit
        sum = sum + mod;    // add to sum
        n = n / 10;         // remove last digit
    }
    return sum;
}
```

> **Example:** n = 1234 → sum = 4+3+2+1 = **10**

---

020 — Sum of Even Digits Only

```
public static int Add(int n) {
    int sum = 0;
    while (n != 0) {
        int mod = n % 10;
        if (mod % 2 == 0) {   // only add if digit is eve
            sum = sum + mod;
        }
```

```
        n = n / 10;
    }
    return sum;
}
```

> **Example:** n = 1234 → even digits: 4, 2 → sum = **6**

---

021 — Sum of Odd Digits Only

```
public static int Add(int n) {
    int sum = 0;
    while (n != 0) {
        int mod = n % 10;
        if (mod % 2 != 0) {   // only add if digit is odd
            sum = sum + mod;
        }
        n = n / 10;
    }
    return sum;
}
```

> **Example:** n = 1234 → odd digits: 3, 1 → sum = **4**

---

022 — Reverse a Number

```
public static String Reverse(int n) {
    String rev = "";
    while (n != 0) {
        int mod = n % 10;     // get last digit
        rev = rev + mod;      // append to string
        n = n / 10;
    }
    return rev;
}
```

> **Example:** n = 1234 → rev = "4321"
>
> **Note:** This returns a `String`. For pure integer reversal, use `int rev` and do `rev = rev * 10 + mod`.

Key Patterns Summary

Pattern	Code Snippet	Use When
Check divisibility	`n % i == 0`	Factors, Even/Odd
Extract last digit	`n % 10`	Digit problems
Remove last digit	`n / 10`	Digit problems
Optimised prime	`i * i <= n`	Prime check
Count with while	`while (count < target)`	First N primes, Nth prime
GCD subtraction	Subtract smaller from larger	GCD
LCM formula	`(a * b) / gcd(a, b)`	LCM

Concepts Checklist

- ☒ For loops — increment and decrement
- ☒ While loops — condition-based stopping
- ☒ Modulo operator `%` for divisibility
- ☒ Even/Odd detection
- ☒ Factors of a number
- ☒ Optimised prime check using sqrt(n)

- ☒ Reusable helper methods (PrimeCheck, GCD)
- ☒ GCD via Euclidean algorithm
- ☒ LCM via GCD formula
- ☒ Digit extraction with `% 10` and `/ 10`
- ☒ Counting digits
- ☒ Sum/filter digits (even, odd)
- ☒ Reversing a number

Date: Day 01 of 30 Days Unstoppable Challenge